

(20 pts) Questão 1: Um motor industrial trabalha com 3 rotações: 1200 RPM, 1800 RPM e 3600 RPM. Um painel com 4 displays de 7 segmentos mostra o ajuste atual. Implemente um **programa** cíclico que mostre no display qual é a rotação atual: **0000, 1200, 1800, 3600**. A rotação atual é sinalizada por circuitos ligados aos bits 0 e 1 da porta B na forma binária (00 01 10 11). O valor **00** representa que o motor está desligado e deve mostrar o valor 0000. Utilize as bibliotecas config.h, pic18f4520.h e ssd.h para a implementação.

(25 pts) Questão 2: Um medidor de vazão possui um dispositivo eletrônico que acende uma luz verde (Porta B, bit 0) quando a vazão está abaixo de 30m³/s, entre 30m³/s e 35m³/s uma luz vermelha (Porta B, bit 1) e acima disso além da luz vermelha um alarme (Porta A, bit 5). Implemente um **programa** para esse sistema sabendo que a vazão é obtida em metros cúbicos por segundo através da função adcRead() da biblioteca adc.h, esta função retorna zero quando a vazão é zero e o valor retornado corresponde exatamente ao valor medido.

(25 pts) Questão 3: Um automóvel utiliza um sensor ultrassônico para indicar a distância de objetos na traseira do veículo. A indicação é feita por um conjunto de 8 leds ligados a porta A do microcontrolador. Cada led representa uma distância de 30cm. Implemente uma **biblioteca** com as funções displayInit() e displayDistancia() para ser utilizada neste dispositivo. A leitura da distância será feita pelo programa principal e não pela biblioteca.

(30 pts) Questão 4: Um sistema de alerta possui um sensor de pressão com resposta linear ligado a um conversor ADC de um microcontrolador, quando a pressão está acima de 35 PSI uma luz de emergência (bit 3 porta B) deve piscar com intervalos de 3 segundos (3s ligada e 3s desligada). O conversor ADC fornece valores de 0 a 1023 que correspondem a valores de pressão de 0 a 100 PSI. Implemente o **programa** utilizando a interrupção do Timer para controlar o piscar da lâmpada.

```
char flag;
//Esta interrupção é chamada a cada 1ms.
void Int(void) __interrupt() {
    if (BitTst(INTCON, 2)) { //Verifica se ocorreu overflow de TMR0
        timerReset(1000); //Ajusta o contador do TMR0
        BitClr(INTCON, 2); //Desliga o Flag de everflow do TMR0
        flag = 1;
    }
}
```

Obs: Uma função InitInterrupt() ajusta todos os parâmetros da interrupção.

```
//adc.h
void adcInit(void);
int adcRead(void);

//lcd.h
void lcdInit(void);
void lcdChar(char letra);
void lcdCommand(char val);

//pwm.h
void pwmInit(void);
void pwmFrequency(unsigned int freq);
void pwmSet1(unsigned char porcento);

//config.h
Necessário para configurar o hardware
Possui comandos: #pragma ...
```

```
//pic18f4520.h
Define TRISA, PORTA,... bem como:
BitSet(arg,bit) BitClr(arg,bit)
BitFlp(arg,bit) BitTst(arg,bit)

//ssd.h
void ssdInit(void);
void ssdUpdate(void);
void ssdDigit(int val, int pos);

//timer.h
void timerInit(void);
void timerReset(unsigned int tempo);
void timerWait(void);

//Observação: TRIS = 0 → Saída
//Em ASCII caracter '0' = 48
```